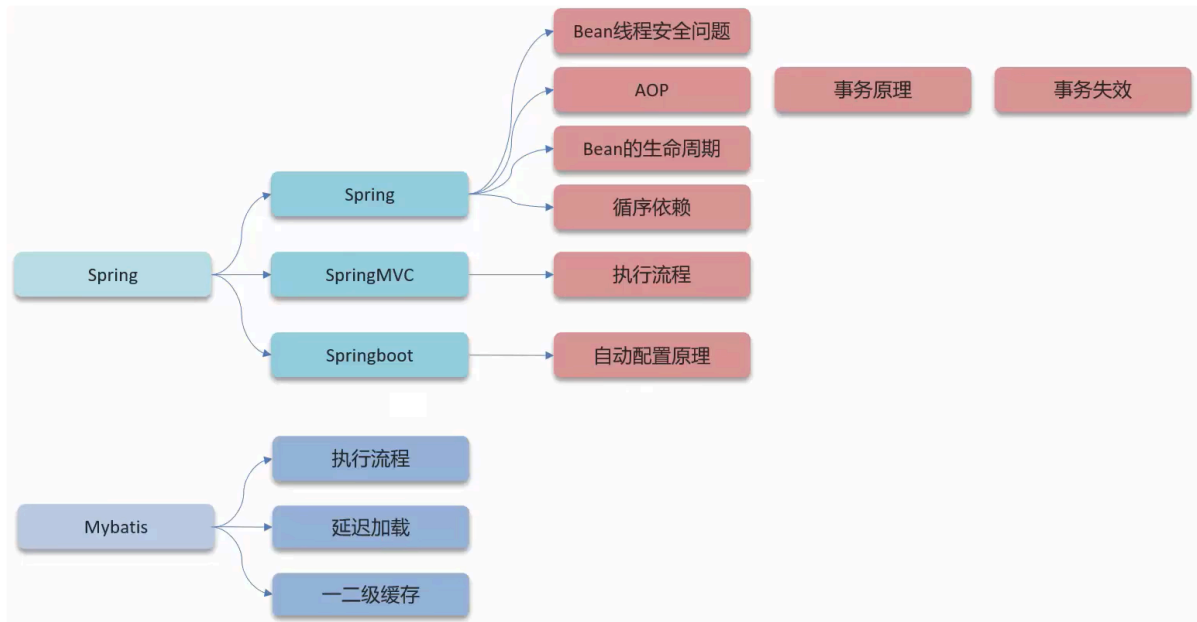


框架篇



Spring

单例bean是线程安全的吗？

Spring框架中的bean是单例的吗？

```
@Service
@Scope("singleton")
public class UserServiceImpl implements UserService {

}
```

- **singleton** (默认)：bean在每个Spring IOC容器中只有一个实例
- **prototype**：一个bean的定义可以有多个实例

不是线程安全的



```
@Controller
@RequestMapping("/user")
public class UserController {

    private int count; 成员方法需考虑线程安全

    @Autowired
    private UserService userService;

    @GetMapping("/getById/{id}")
    public User getById(@PathVariable("id") Integer id){
        count++;
        System.out.println(count);
        return userService.getById(id);
    }
}
```

服务器中的代码片段

Spring bean并没有可变的狀態(比如Service类和DAO类)，所以在某种程度上说Spring的单例bean是线程安全的。

面试官：Spring框架中的单例bean是线程安全的吗？

答：不是线程安全的

Spring框架中有一个@Scope注解，默认的值就是 singleton，单例的。

因为一般在Spring的bean中都是注入无状态的对象，没有线程安全问题，但是如果在bean中定义了可修改的成员变量，是要考虑线程安全问题的，可以使用多例或者加锁来解决

AOP

AOP称为面向切面编程，用于将那些与业务无关，但却对多个对象产生影响的公共行为和逻辑，抽取并封装为一个可重用的模块，这个模块被命名为“切面”(Aspect)，减少系统中的重复代码，降低了模块间的耦合度，同时提高了系统的可维护性。

常见的AOP使用场景：

- 记录操作日志
- 缓存处理
- Spring中内置的事务处理

记录操作日志思路

☐	用户名	请求方式	访问地址	模块名称	登录IP	操作时间	操作
☐	admin	GET	/monitor/operlog/export	操作日志	172.17.32.253	2023-02-06 15:29:19	查看
☐	admin	PUT	/clues/clue/gain	批量捞取	172.17.32.253	2023-02-06 14:55:50	查看
☐	admin	PUT	/clues/clue/false/9003	伪线索	172.17.32.253	2023-02-06 14:55:41	查看
☐	admin	PUT	/clues/clue/false/9002	伪线索	172.17.32.253	2023-02-06 14:55:32	查看
☐	admin	PUT	/clues/clue/false/9031	伪线索	172.17.32.253	2023-02-06 14:55:28	查看

获取请求的用户名、请求方式、访问地址、模块名称、登录ip、操作时间，记录到数据库的日志表中



实例代码

```
1  @Component
2  @Aspect    //切面类
3  public class SysAspect {
4
5      @Pointcut("@annotation(com.itheima.annotation.Log)")
6      private void pointcut() {
7
8      }
9
10     @Pointcut("execution(* com.itheima.service.*(..))")
11     public void pointcut2(){
12
13     }
14     @Around("pointcut2()")
15     public Object around(ProceedingJoinPoint joinPoint) throws Throwable {
16         //获取用户名
17         //需要通过解析session或token获取
18
19         //获取被增强类和方法的信息
20         Signature signature = joinPoint.getSignature();
21         MethodSignature methodSignature = (MethodSignature) signature;
22         //获取被增强的方法对象
23         Method method = methodSignature.getMethod();
24         //从方法中解析注解
25         if(method != null){
26             Log logAnnotation = method.getAnnotation(Log.class);
27             System.out.println(logAnnotation.name());
28         }
29         //方法名字
30         String name = method.getName();
31         System.out.println(name);
32
33         //通过工具类获取Request对象
34         RequestAttributes reqa =
35             RequestContextHolder.getRequestAttributes();
36         ServletRequestAttributes sra = (ServletRequestAttributes)reqa;
37         HttpServletRequest request = sra.getRequest();
38         //访问的url
39         String url = request.getRequestURI().toString();
40         System.out.println(url);
41         //请求方式
42         String methodName = request.getMethod();
```

```

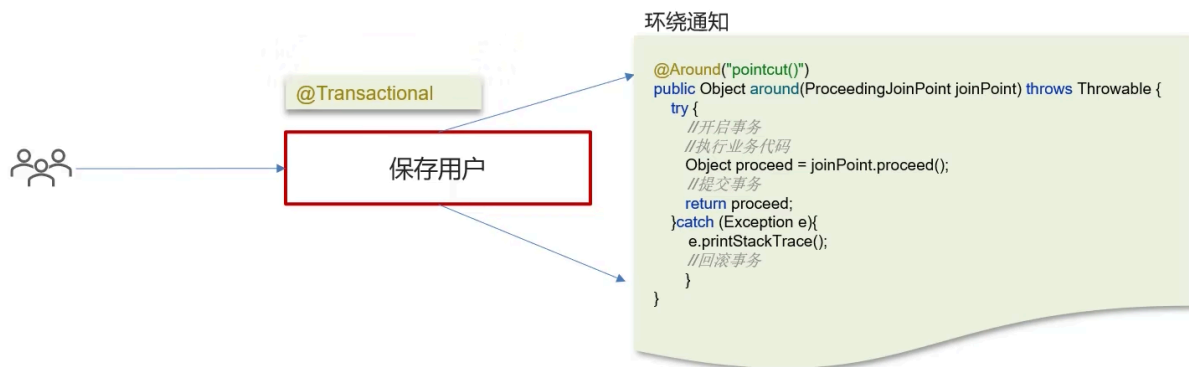
41         System.out.println(methodName);
42
43         //登录IP
44         String ipAddr = getIpAddr(request);
45         System.out.println(ipAddr);
46
47         //操作时间
48         System.out.println(new Date());
49
50         //保存到数据库（操作日志）
51         //....
52
53         return joinPoint.proceed();
54     }
55
56     /**
57      * 获取ip地址
58      * @param request
59      * @return
60      */
61     public String getIpAddr(HttpServletRequest request){
62         String ip = request.getHeader("x-forwarded-for");
63         if(ip == null || ip.length() == 0 ||
"unknown".equalsIgnoreCase(ip)){
64             ip = request.getHeader("Proxy-Client-IP");
65         }
66         if(ip == null || ip.length() == 0 ||
"unknown".equalsIgnoreCase(ip)){
67             ip = request.getHeader("WL-Proxy-Client-IP");
68         }
69         if(ip == null || ip.length() == 0 ||
"unknown".equalsIgnoreCase(ip)){
70             ip = request.getRemoteAddr();
71         }
72
73         return "0:0:0:0:0:0:0:1".equals(ip) ? "127.0.0.1" : ip;
74     }
75 }

```

Spring中的事务是如何实现的

Spring支持编程式事务管理和声明式事务管理两种方式。

- 编程式事务控制：需要使用 **TransactionTemplate** 来进行实现，对业务代码有侵入性，项目中很少使用
- 声明式事务管理：声明式事务管理是建立在AOP之上的。其本质是通过AOP功能，对方法前后进行拦截，对事务处理的功能编织到拦截的方法中，也就是在目标方法开始之前加入一个事务，在执行完目标方法之后根据执行情况提交或者回滚事务。



面试官：什么是AOP

答：面向切面编程，用于将那些与业务无关，但却对多个对象产生影响的公共行为和逻辑，抽取公共模块复用，降低耦合

面试官：你们项目中有没有使用到AOP

记录操作日志，缓存，Spring实现的事务

核心是：使用AOP中的环绕通知 + 切点表达式(要找到记录日志的方法)，通过环绕通知的参数获取请求方法的参数(类、方法、注解、请求方式等)，获取到这些参数以后，保存到数据库

面试官：Spring中的事务是如何实现的

其本质是通过AOP功能，对方法前后进行拦截，在执行方法之前开启事务，在执行完目标方法之后根据执行情况提交或者回滚事务。

事务失效的场景

- 异常捕获处理
- 抛出检查异常
- 非public方法

异常捕获处理

```
@Transactional
public void update(Integer from, Integer to, Double money) {
    try {
        //转账的用户不能为空
        Account fromAccount = accountDao.selectById(from);
        //判断用户的钱是否够转账
        if (fromAccount.getMoney() - money >= 0) {
            fromAccount.setMoney(fromAccount.getMoney() - money);
            accountDao.updateById(fromAccount);

            //异常
            int a = 1/0;

            //被转账的用户
            Account toAccount = accountDao.selectById(to);
            toAccount.setMoney(toAccount.getMoney() + money);
            accountDao.updateById(toAccount);
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

原因：事务通知只有捕捉到了目标抛出的异常，才能进行后续的回滚处理，如果目标自己处理掉异常，事务通知无法知悉

解决方法：在 catch 块添加 `throw new RuntimeException(e)` 抛出

抛出检查异常

```
@Transactional
public void update(Integer from, Integer to, Double money) throws FileNotFoundException {
    //转账的用户不能为空
    Account fromAccount = accountDao.selectById(from);
    //判断用户的钱是否够转账
    if (fromAccount.getMoney() - money >= 0) {
        fromAccount.setMoney(fromAccount.getMoney() - money);
        accountDao.updateById(fromAccount);
        //读取文件
        new FileInputStream("dddd");
        //被转账的用户
        Account toAccount = accountDao.selectById(to);
        toAccount.setMoney(toAccount.getMoney() + money);
        accountDao.updateById(toAccount);
    }
}
```

原因：Spring默认只会回滚非检查异常

解决方法：配置 `rollbackFor` 属性

```
1 | @Transactional(rollbackFor=Exception.class)
```

非public方法导致的事务失效

```
@Transactional(rollbackFor = Exception.class)
void update(Integer from, Integer to, Double money) throws FileNotFoundException {
    //转账的用户不能为空
    Account fromAccount = accountDao.selectById(from);
    //判断用户的钱是否够转账
    if (fromAccount.getMoney() - money >= 0) {
        fromAccount.setMoney(fromAccount.getMoney() - money);
        accountDao.updateById(fromAccount);

        //读取文件
        new FileInputStream("dddd");

        //被转账的用户
        Account toAccount = accountDao.selectById(to);
        toAccount.setMoney(toAccount.getMoney() + money);
        accountDao.updateById(toAccount);
    }
}
```

原因：Spring为方法创建代理、添加事务通知，前提条件都是该方法是 **public** 的

解决方法：改为 **public** 方法

面试官：Spring中事务失效的场景有哪些？

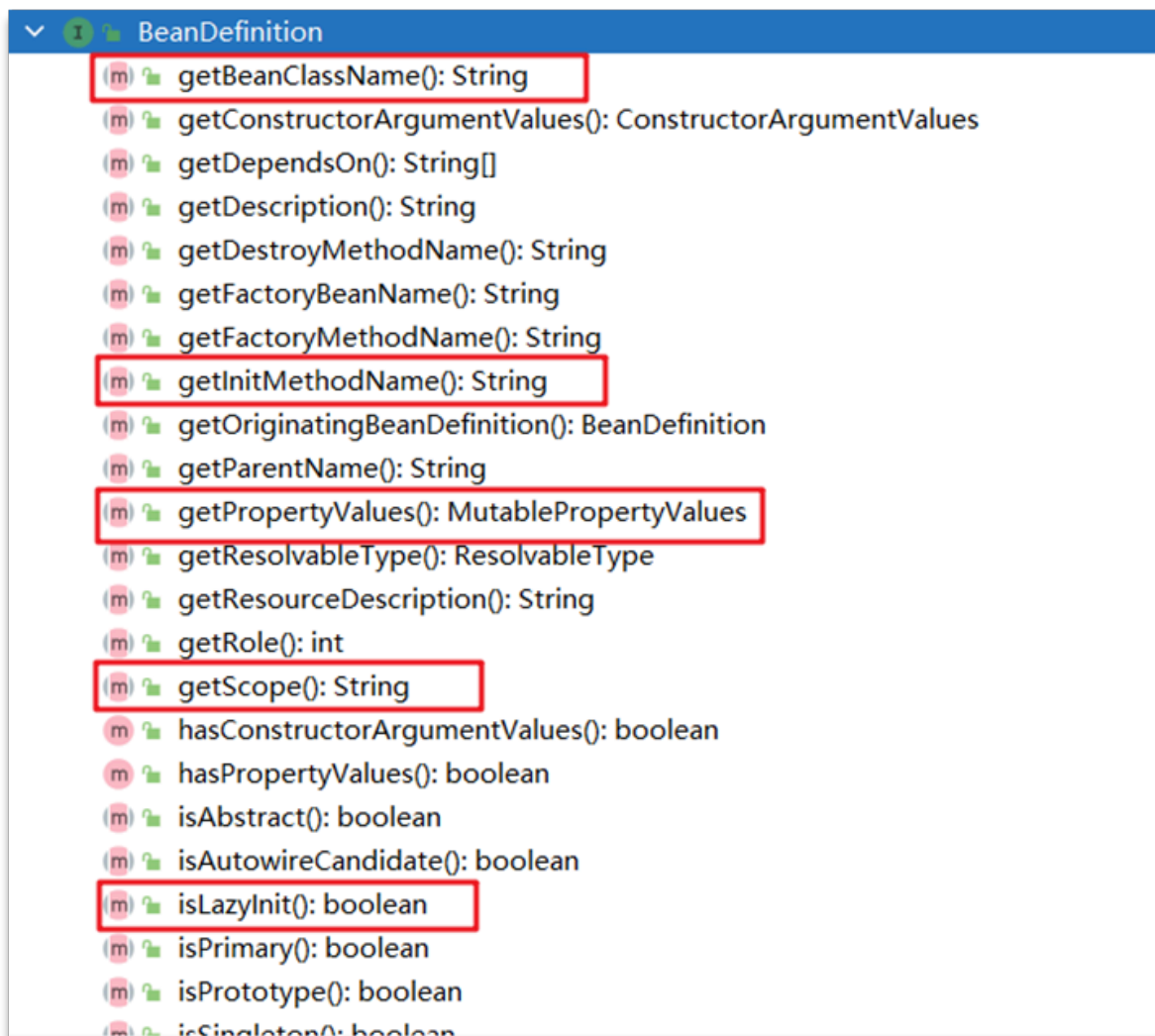
1. 异常捕获处理，自己处理了异常，没有抛出导致的事务失效，解决方案：手动抛出异常
2. 抛出检查异常，解决方案：配置 **rollbackFor** 属性为 **Exception**
3. 非 **public** 方法导致的事务失效，解决方案：改为 **public**

bean的生命周期

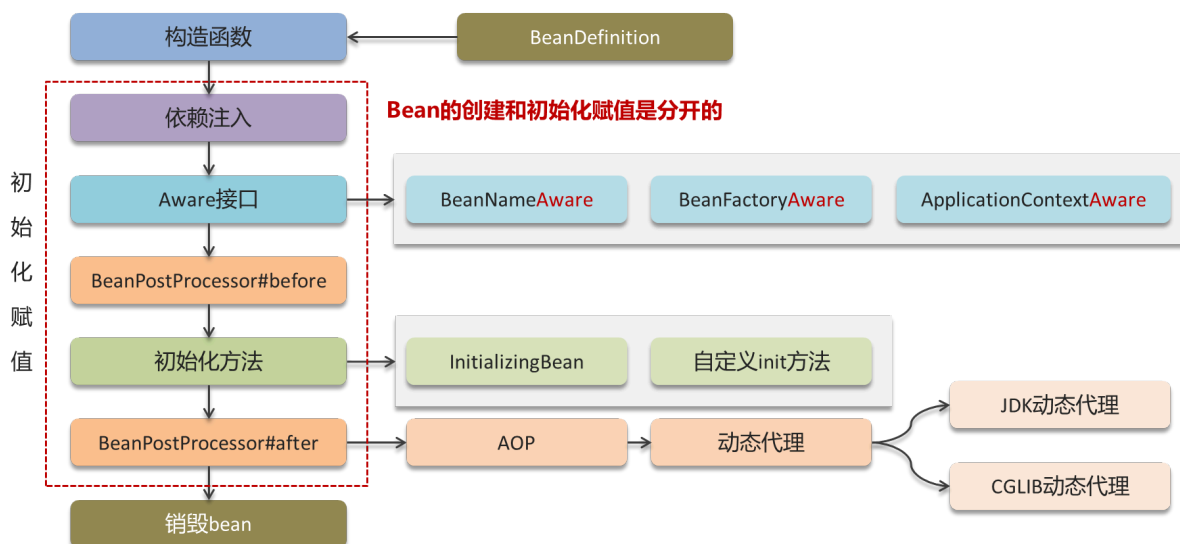
BeanDefinition

Spring容器在进行实例化时，会将 **xml** 配置的 **<bean>** 的信息封装成一个 **BeanDefinition** 对象，Spring根据 **BeanDefinition** 来创建 **Bean** 对象，里面有很多的属性用来描述 **Bean**

```
<bean id="userDao" class="com.itheima.dao.impl.UserDaoImpl" lazy-init="true"/>
<bean id="userService" class="com.itheima.service.UserServiceImpl" scope="singleton">
    <property name="userDao" ref="userDao"></property>
</bean>
```



- **beanClassName** : bean 的类名
- **initMethodName** : 初始化方法名称
- **propertyValues** : bean 的属性值
- **scope** : 作用域
- **lazyInit** : 延迟初始化

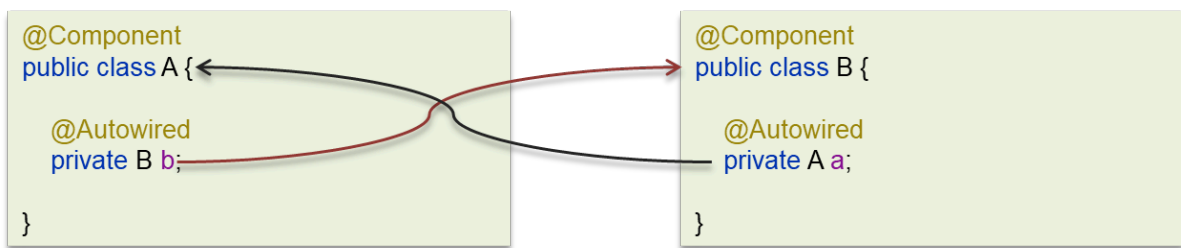


面试官：Spring的bean的生命周期

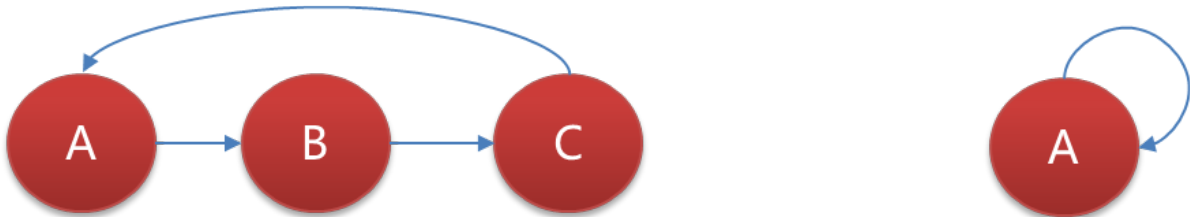
答：

1. 通过 `BeanDefinition` 获取 `bean` 的定义信息
2. 调用构造函数实例化 `bean`
3. `bean` 的依赖注入
4. 处理 `Aware` 接口(`BeanNameAware`、`BeanFactoryAware`、`ApplicationContextAware`)
5. `Bean` 的后置处理器 `BeanPostProcessor` - 前置
6. 初始化方法(`InitializingBean`、`init-method`)
7. `Bean` 的后置处理器 `BeanPostProcessor` - 后置
8. 销毁 `bean`

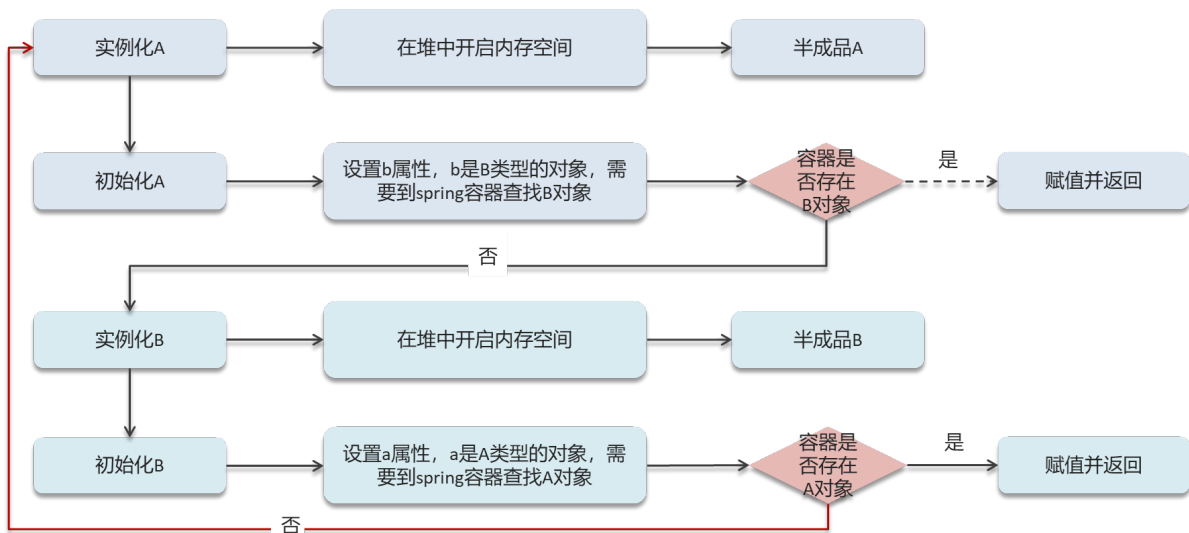
Bean的循环依赖(循环引用)



在创建A对象的同时需要使用到B对象，在创建B对象的同时需要使用到A对象



什么是Spring的循环依赖?



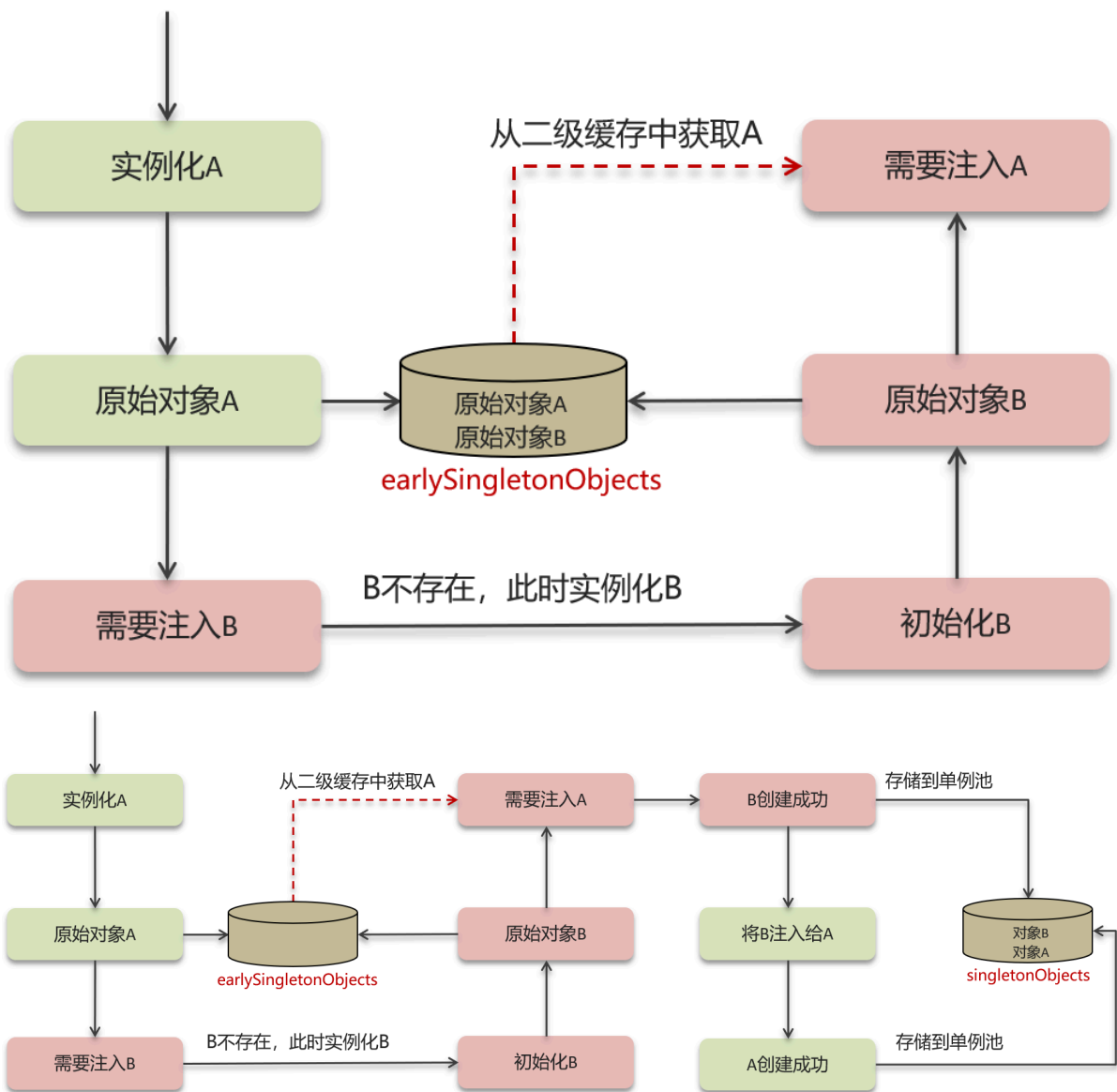
Spring解决循环依赖是通过三级缓存，对应的三级缓存如下所示：

```
public class DefaultSingletonBeanRegistry extends SimpleAliasRegistry implements SingletonBeanRegistry {
    private static final int SUPPRESSED_EXCEPTIONS_LIMIT = 100;

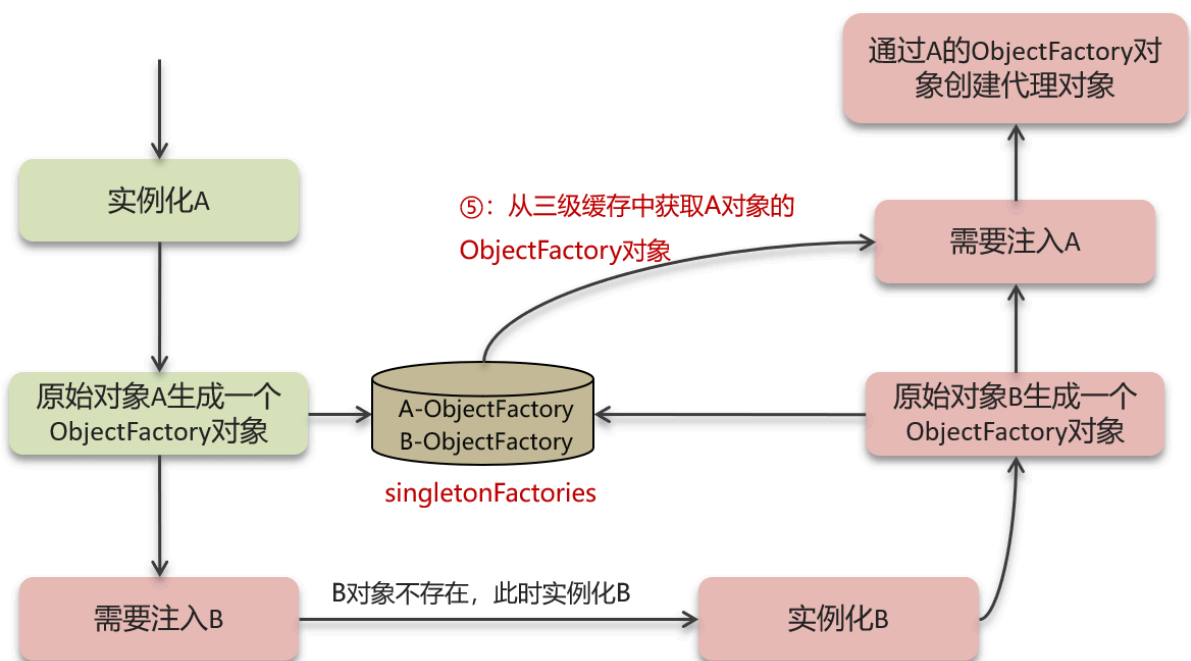
    private final Map<String, Object> singletonObjects = new ConcurrentHashMap(256);           一级缓存
    private final Map<String, ObjectFactory<?>> singletonFactories = new HashMap(16);       三级缓存
    private final Map<String, Object> earlySingletonObjects = new ConcurrentHashMap(16);     二级缓存
}
```

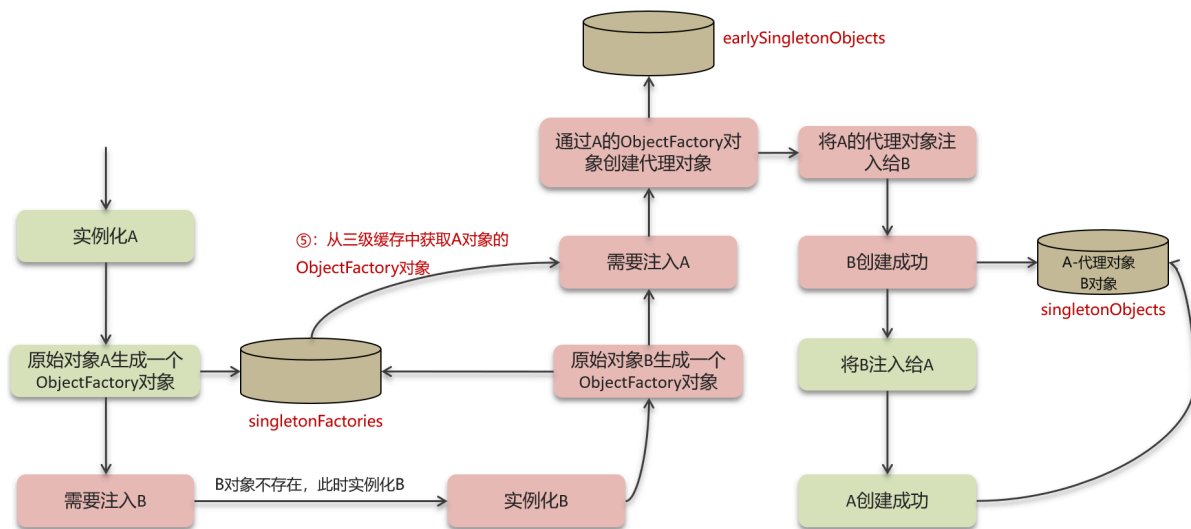
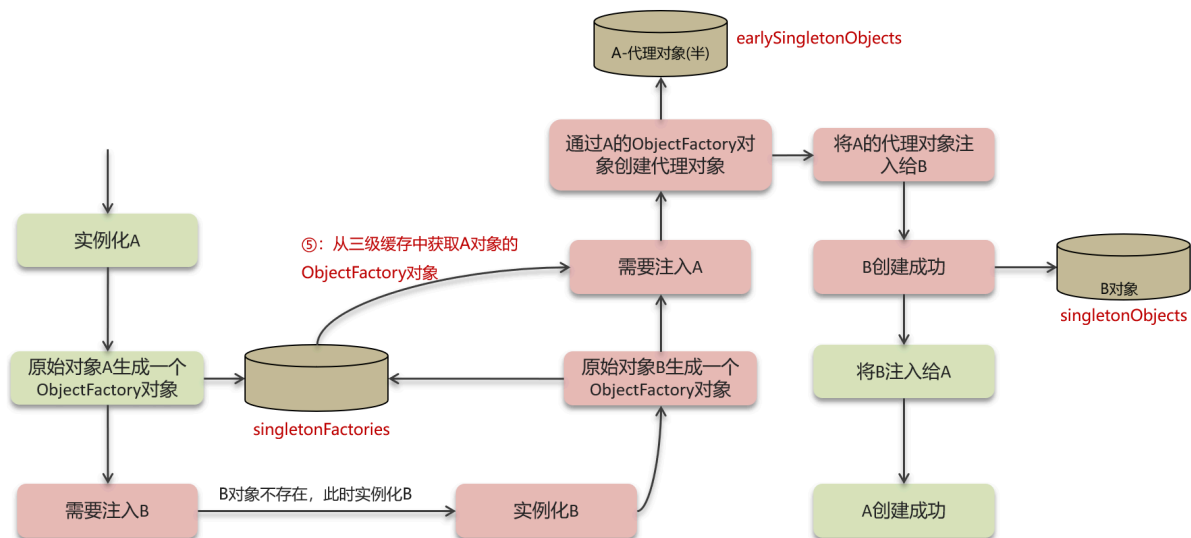
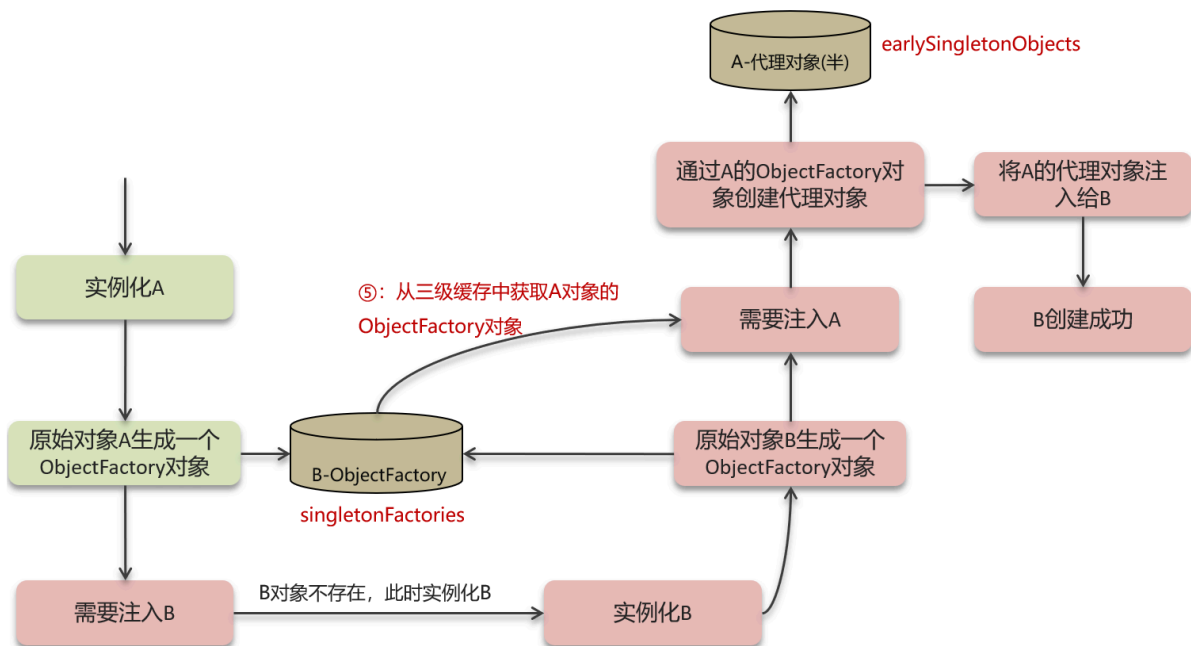
一级缓存作用：限制 bean 在 beanFactory 中只存一份，即实现 singleton scope，解决不了循环依赖





一级和二级缓存可以解决一般对象的循环依赖，如果一个对象被增强了，即成为了代理对象，这时候就需要借助三级缓存了





构造方法出现了循环依赖

```
@Component
public class A {

    // B成员变量
    private B b;

    public A(B b){
        System.out.println("A的构造方法执行了...");
        this.b = b ;
    }
}
```

```
@Component
public class B {

    // A成员变量
    private A a;

    public B(A a){
        System.out.println("B的构造方法执行了...");
        this.a = a ;
    }
}
```

报错信息：

Is there an unresolvable circular reference?

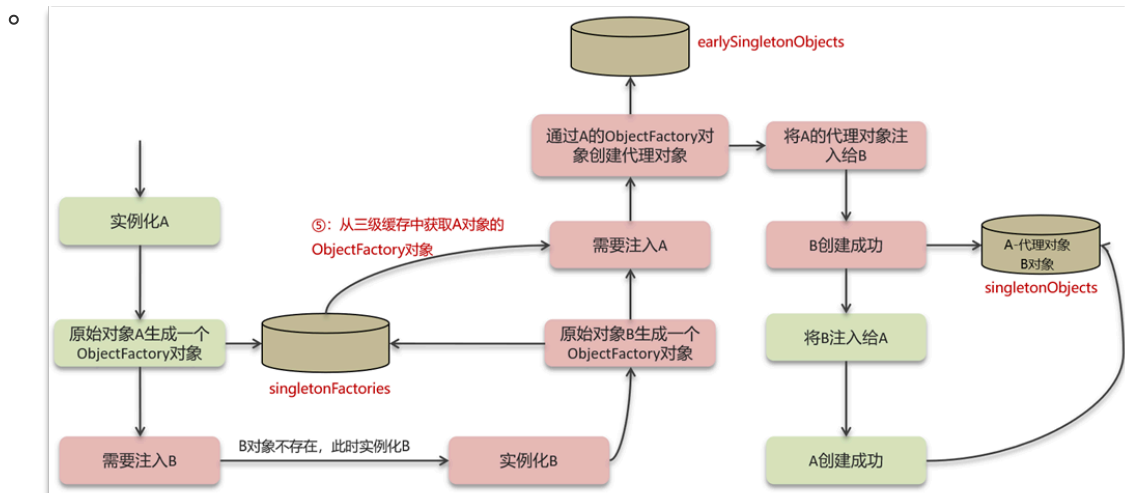
解决方法：

```
public A(@Lazy B b){
    System.out.println("A的构造方法执行了...");
    this.b = b ;
}
```

@Lazy 代表暂时不注入，什么时候用到什么时候再去实例化对象

面试官：Spring中的循环引用

- 循环依赖：循环依赖其实就是循环引用，也就是两个或两个以上的 **bean** 互相持有对方，最终形成闭环。比如A依赖于B，B依赖于A
- 循环依赖在Spring中是允许存在的，Spring框架依据三级缓存已经解决了大部分的循环依赖
 - 一级缓存：单例池，缓存已经经历了完整的生命周期，已经初始化完成的 **bean** 对象
 - 二级缓存：缓存早期的 **bean** 对象(生命周期还没走完)
 - 三级缓存：缓存的事 **ObjectFactory**，表示对象工厂，用来创建某个对象的

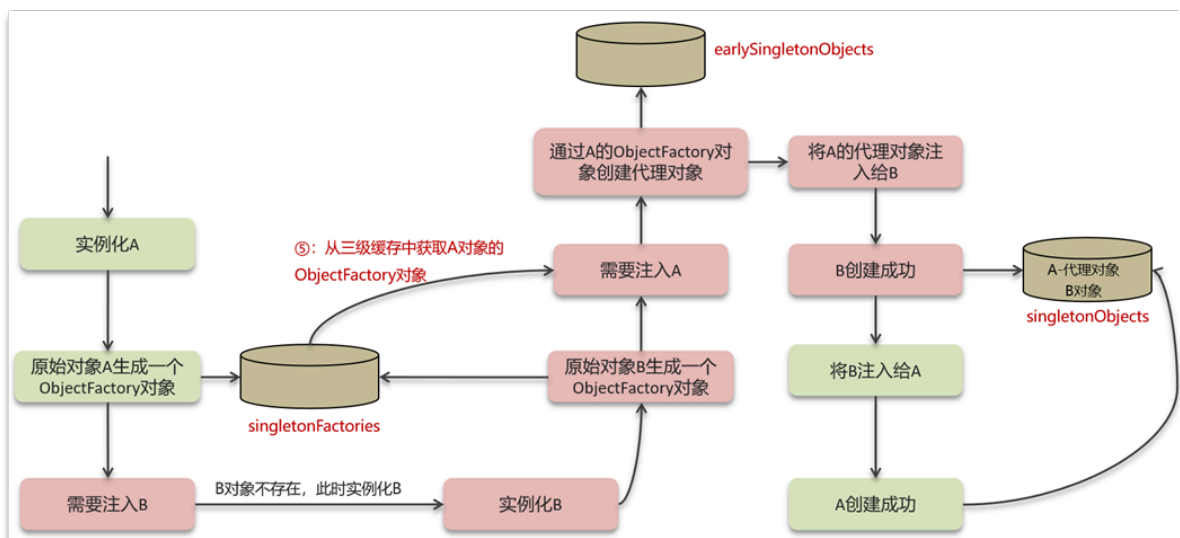


面试官：构造方法出现了循环依赖怎么解决？

答：A依赖于B，B依赖于A，注入的方式是构造函数

原因：由于 **bean** 的生命周期中构造函数是第一个执行的，Spring框架的三级缓存并不能解决构造函数的依赖注入

解决方案：使用 **@Lazy** 进行懒加载，什么时候需要对象再进行 **bean** 对象的创建



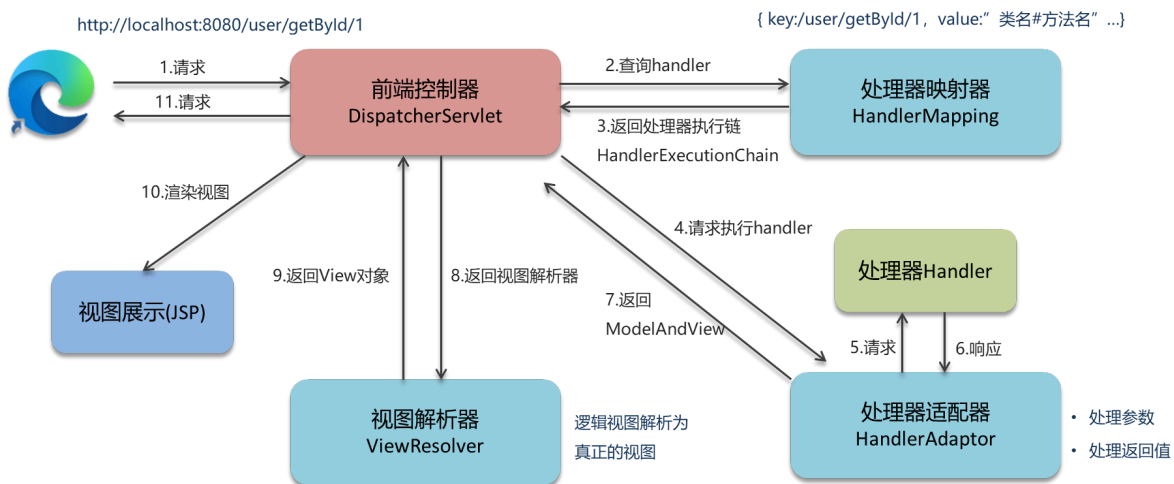
SpringMVC

执行流程

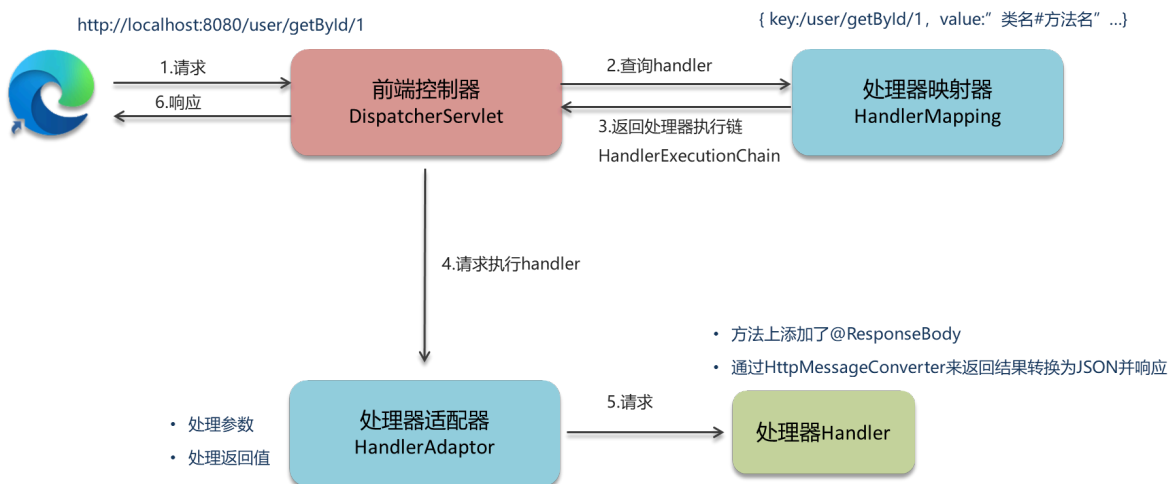
SpringMVC的执行流程是这个框架最核心的内容

- 视图阶段(老旧JSP等)
- 前后端分离阶段(接口开发, 异步)

视图阶段(JSP)



前后端分离阶段(接口开发, 异步请求)



面试官：SpringMVC的执行流程知道吗

答：(版本1：视图版本, JSP)

1. 用户发送出请求到前端控制器 **DispatcherServlet**
2. **DispatcherServlet** 收到请求调用 **HandlerMapping** (处理器映射器)
3. **HandlerMapping** 找到具体的处理器, 生成处理器对象及处理器拦截器(如果有), 再一起返回给 **DispatcherServlet**
4. **DispatcherServlet** 调用 **HandlerAdapter** (处理器适配器)
5. **HandlerAdapter** 经过适配调用具体的处理器(**Handler** / **Controller**)

6. **Controller** 执行完成返回 **ModelAndView** 对象
7. **HandlerAdapter** 将 **Controller** 执行结果 **ModelAndView** 返回给 **DispatcherServlet**
8. **DispatcherServlet** 将 **ModelAndView** 传给 **ViewResolver** (视图解析器)
9. **ViewResolver** 解析后返回具体 **View** (视图)
10. **DispatcherServlet** 根据 **View** 进行渲染视图(即将模型数据填充至视图中)
11. **DispatcherServlet** 响应用户

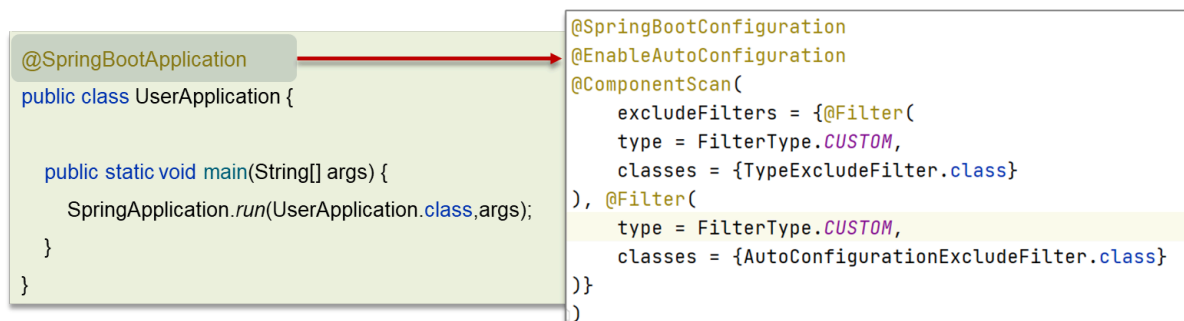
(版本2: 前后端开发, 接口开发)

1. 用户发送出请求到前端控制器 **DispatcherServlet**
2. **DispatcherServlet** 收到请求调用 **HandlerMapping** (处理器映射器)
3. **HandlerMapping** 找到具体的处理器, 生成处理器对象及处理器拦截器(如果有), 再一起返回给 **DispatcherServlet**
4. **DispatcherServlet** 调用 **HandlerAdapter** (处理器适配器)
5. **HandlerAdapter** 经过适配调用具体的处理器(**Handler** / **Controller**)
6. 方法上添加了 **@ResponseBody**
7. 通过 **HttpMessageConverter** 来将返回结果转换为 **JSON** 并响应

SpringBoot

自动配置

SpringBoot中最高频的一道面试题, 也是框架最核心的思想



- **@SpringBootApplication**: 该注解与 **@Configuration** 注解作用相同, 用来声明当前也是一个配置类.
- **ComponentScan**: 组件扫描, 默认扫描当前引导类所在包及其子包.
- **EnableAutoConfiguration**: SpringBoot实现自动化配置的核心注解


```

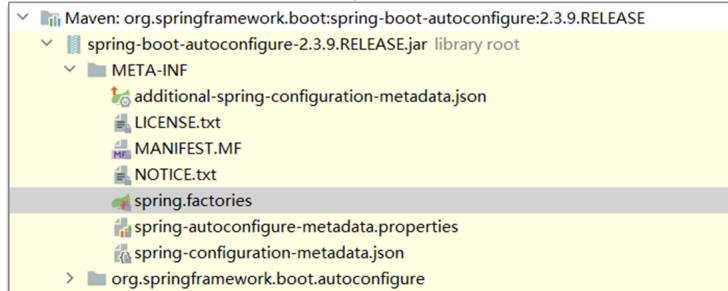
@Target({ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Inherited
@AutoConfigurationPackage
@Import({AutoConfigurationImportSelector.class})
public @interface EnableAutoConfiguration {

```

```

@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan(
    excludeFilters = {@Filter(
        type = FilterType.CUSTOM,
        classes = {TypeExcludeFilter.class}
    )}, @Filter(
        type = FilterType.CUSTOM,
        classes = {AutoConfigurationExcludeFilter.class}
    )
)

```



spring.factories:

```

# Auto Configure
org.springframework.boot.autoconfigure.EnableAutoConfiguration=\
org.springframework.boot.autoconfigure.admin.SpringApplicationAdminJmxAutoConfiguration,\
org.springframework.boot.autoconfigure.aop.AopAutoConfiguration,\
org.springframework.boot.autoconfigure.amqp.RabbitAutoConfiguration,\
org.springframework.boot.autoconfigure.batch.BatchAutoConfiguration,\
org.springframework.boot.autoconfigure.cache.CacheAutoConfiguration,\
org.springframework.boot.autoconfigure.cassandra.CassandraAutoConfiguration,\
org.springframework.boot.autoconfigure.context.ConfigurationPropertiesAutoConfiguration,\
org.springframework.boot.autoconfigure.context.LifecycleAutoConfiguration,\
org.springframework.boot.autoconfigure.context.MessageSourceAutoConfiguration,\
org.springframework.boot.autoconfigure.context.PropertyPlaceholderAutoConfiguration,\
org.springframework.boot.autoconfigure.couchbase.CouchbaseAutoConfiguration,\
org.springframework.boot.autoconfigure.dao.PersistenceExceptionTranslationAutoConfiguration,\
org.springframework.boot.autoconfigure.data.cassandra.CassandraDataAutoConfiguration,\
org.springframework.boot.autoconfigure.data.cassandra.CassandraReactiveDataAutoConfiguration,\
org.springframework.boot.autoconfigure.data.cassandra.CassandraReactiveRepositoriesAutoConfiguration,\
org.springframework.boot.autoconfigure.data.cassandra.CassandraRepositoriesAutoConfiguration,\
org.springframework.boot.autoconfigure.data.couchbase.CouchbaseDataAutoConfiguration,\

```

RedisAutoConfiguration

```

@Configuration(
    proxyBeanMethods = false  是一个配置类
)
@ConditionalOnClass({RedisOperations.class}) 判断是否有对应字节码
@EnableConfigurationProperties({RedisProperties.class})
@Import({LettuceConnectionConfiguration.class, JedisConnectionConfiguration.class})
public class RedisAutoConfiguration {
    public RedisAutoConfiguration() {
    }

    @Bean
    @ConditionalOnMissingBean(
        name = {"redisTemplate"} 判断环境中没有对应的bean
    )
    public RedisTemplate<Object, Object> redisTemplate(RedisConnectionFactory redisCo
        RedisTemplate<Object, Object> template = new RedisTemplate();
        template.setConnectionFactory(redisConnectionFactory);
        return template;
    }
}

```

面试官：SpringBoot自动配置原理

答：

1. 在SpringBoot项目中的引导类上有一个注解 `@SpringBootApplication`，这个注解是对三个注解进行了封装，分别是：
 - `@SpringBootConfiguration`
 - `@EnableAutoConfiguration`
 - `@ComponentScan`
2. 其中 `@EnableAutoConfiguration` 是实现自动化配置的核心注解。该注解通过 `@Import` 注解导入对应的配置选择器

内部就是读取了该项目和该项目引用的Jar包的 `classpath` 路径下 `META-INF/spring.factories` 文件中的所配置的类的全类名。在这些配置类中所定义的Bean会根据条件注解所指定的条件来决定是否需要将其导入到Spring容器中。
3. 条件判断会有像 `@ConditionalOnClass` 这样的注解，判断是否有对应的 `class` 文件，如果有则加载该类，把这个配置类的所有的Bean放入Spring容器中使用

Spring框架常见注解

- Spring的常见注解有哪些？
- SpringMVC常见的注解有哪些？
- SpringBoot常见注解有哪些？

Spring

注解	说明
<code>@Component</code> 、 <code>@Controller</code> 、 <code>@Service</code> 、 <code>@Repository</code>	使用在类上用于实例化Bean
<code>@Autowired</code>	使用在字段上用于根据类型依赖注入
<code>@Qualifier</code>	结合 <code>@Autowired</code> 一起使用用于根据名称进行依赖注入
<code>@Scope</code>	标注Bean的作用范围
<code>@Configuration</code>	指定当前类是一个 Spring 配置类，当创建容器时会从该类上加载注解
<code>@ComponentScan</code>	用于指定 Spring 在初始化容器时要扫描的包
<code>@Bean</code>	使用在方法上，标注将该方法的返回值存储到Spring容器中
<code>@Import</code>	使用 <code>@Import</code> 导入的类会被Spring加载到IOC容器中
<code>@Aspect</code> 、 <code>@Before</code> 、 <code>@After</code> 、 <code>@Around</code> 、 <code>@Pointcut</code>	用于切面编程（AOP）

SpringMVC

注解	说明
<code>@RequestMapping</code>	用于映射请求路径，可以定义在类上和方法上。用于类上，则表示类中的所有的方法都是以该地址作为父路径
<code>@RequestBody</code>	注解实现接收http请求的json数据，将json转换为java对象
<code>@RequestParam</code>	指定请求参数的名称
<code>@PathVariable</code>	从请求路径下中获取请求参数(<code>/user/{id}</code>)，传递给方法的形式参数
<code>@ResponseBody</code>	注解实现将controller方法返回对象转化为json对象响应给客户端
<code>@RequestHeader</code>	获取指定的请求头数据
<code>@RestController</code>	<code>@Controller</code> + <code>@ResponseBody</code>

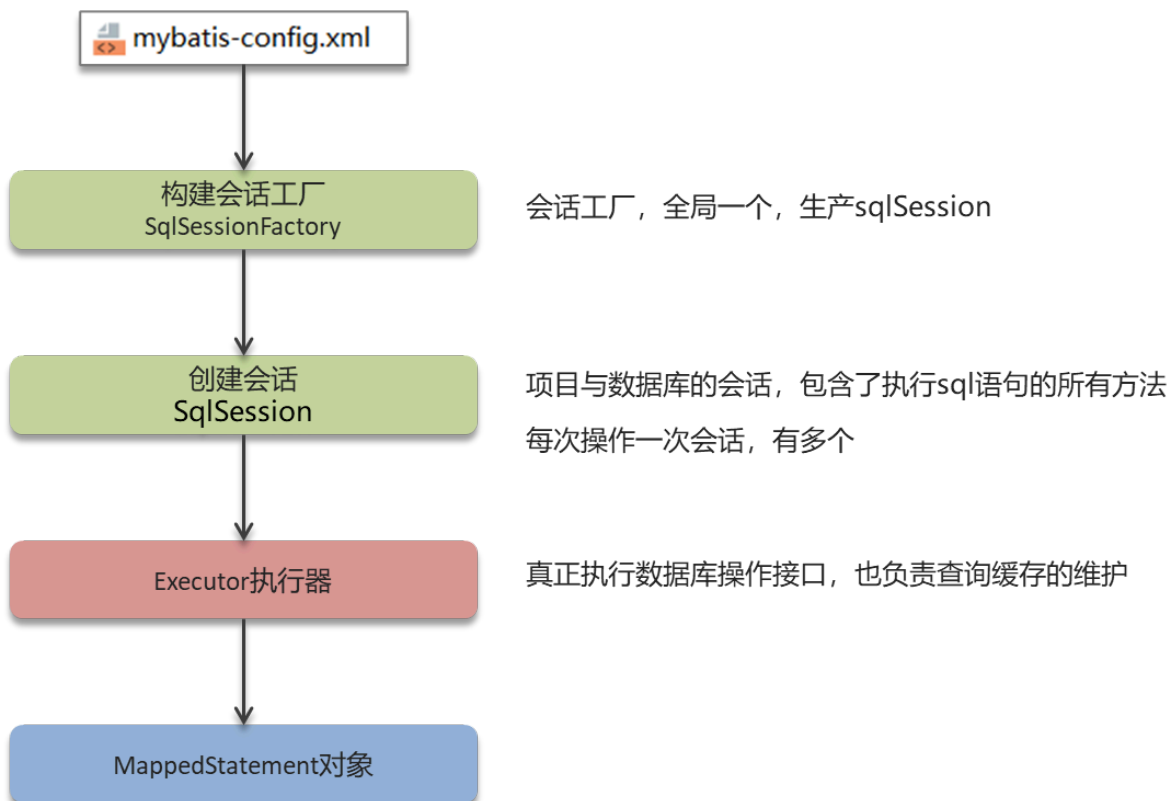
SpringBoot

注解	说明
<code>@SpringBootConfiguration</code>	组合了- <code>@Configuration</code> 注解，实现配置文件的功能
<code>@EnableAutoConfiguration</code>	打开自动配置的功能，也可以关闭某个自动配置的选
<code>@ComponentScan</code>	Spring组件扫描

MyBatis

执行流程

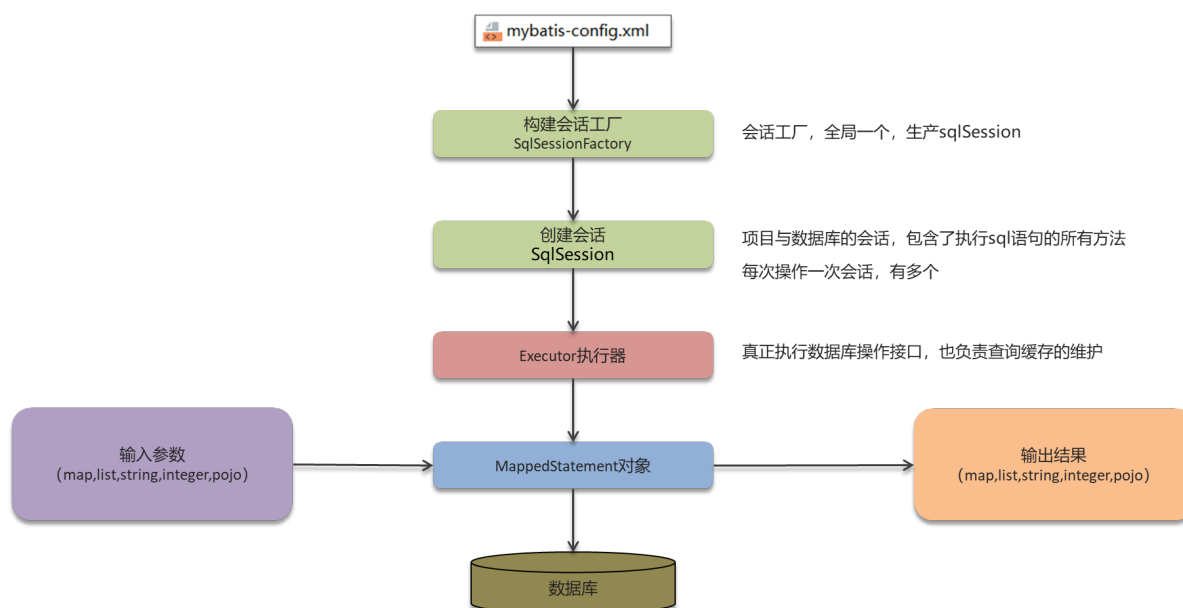
- 理解了各个组件的关系
- SQL的执行过程(参数映射、SQL解析、执行和结果处理)



MappedStatement对象

```
<select id="selectById" resultType="user">
  select * from tb_user where id = #{id};
</select>
```

```
> this = {SimpleExecutor@1961}
> ms = {MappedStatement@1957}
  > resource = "com/itheima/mapper/UserMapper.xml"
  > configuration = {Configuration@1967}
  > id = "com.itheima.mapper.UserMapper.selectById"
    > fetchSize = null
    > timeout = null
  > statementType = {StatementType@1970} "PREPARED"
  > resultSetType = {ResultSetType@1971} "DEFAULT"
  > sqlSource = {RawSqlSource@1972}
    > sqlSource = {StaticSqlSource@2253}
      > sql = "select * from tb_user where id = ?;"
      > parameterMappings = {ArrayList@2255} size = 1
      > configuration = {Configuration@1967}
    > cache = null
  > parameterMap = {ParameterMap@1973}
  > resultMaps = {Collections$UnmodifiableRandomAccessList@1974} size = 1
  > flushCacheRequired = false
  > useCache = true
  > resultOrdered = false
  > sqlCommandType = {SqlCommandType@1975} "SELECT"
  > keyGenerator = {NoKeyGenerator@1976}
  > keyProperties = null
```



面试官：MyBatis执行流程

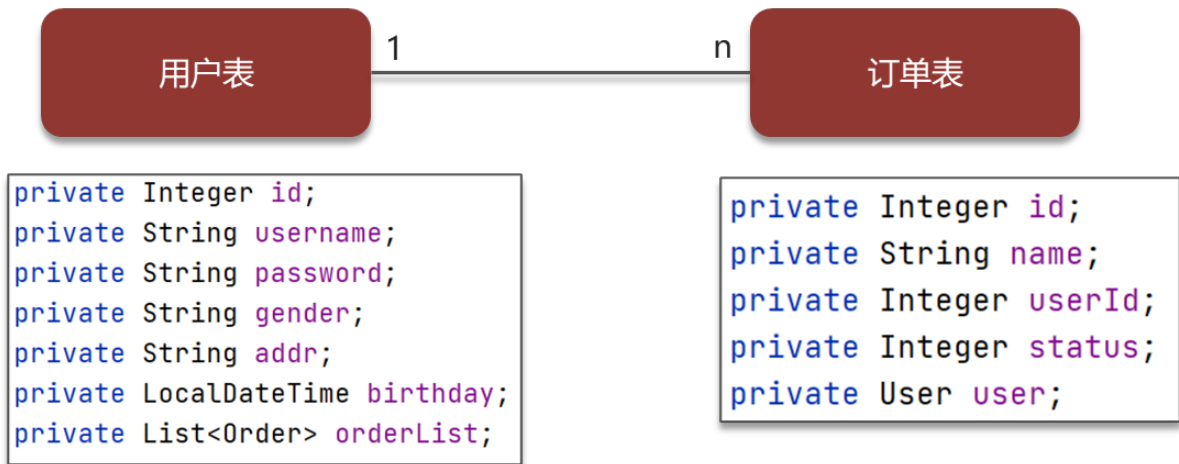
答：

1. 读取MyBatis配置文件： `mybatis-config.xml` 加载运行环境和映射文件
2. 构造会话工厂 `SqlSessionFactory`
3. 会话工厂创建 `SqlSession` 对象(包含了执行SQL语句的所有方法)
4. 操作数据库的接口， `Executor` 执行器，同时负责查询缓存的维护
5. `Executor` 接口的执行方法中有一个 `MappedStatement` 类型的参数，封装了映射信息
6. 输入参数映射
7. 输出结果映射

延迟加载使用及原理

MyBatis支持延迟加载，但默认没有开启

延迟加载



查询用户的时候，把用户所属的订单数据页查询出来，这个是立即加载

查询用户的时候，暂时不查询订单数据，当需要订单的时候，再查询订单，这个就是延迟加载

代码示例

```
1 <resultMap id="userResultMap" type="user" autoMapping="true">
2   <id property="id" column="id"></id>
3   <result property="username" column="username"></result>
4   <result property="password" column="password"></result>
5   <result property="gender" column="gender"></result>
6   <result property="addr" column="addr"></result>
7   <collection property="orderList" ofType="order"
8     select="com.itheima.mapper.OrderMapper.findById"
9     column="id" fetchType="lazy">
10  </collection>
11 </resultMap>
12 <select id="selectById" resultMap="userResultMap">
13   select *
14   from tb_user
15   where id = #{id}
16 </select>
```

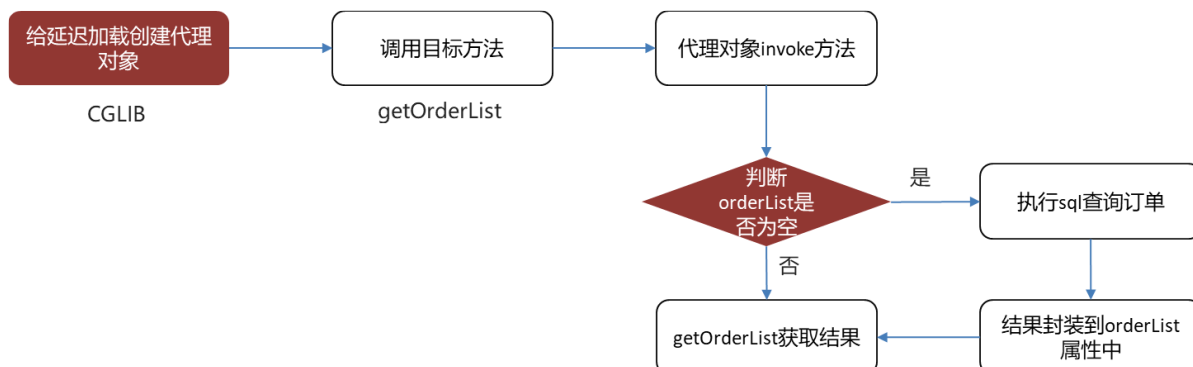
`fetchType="lazy"` 开启延迟加载

同时可以在 `mybatis-config.xml` 中设置全局开启

```
1 <settings>
2   <setting name="cacheEnabled" value="true" /> <!-- 开启二级缓存 -->
3   <setting name="mapUnderscoreToCamelCase" value="true" /> <!-- 开启驼峰命名
   自动映射 -->
4   <setting name="lazyLoadingEnabled" value="true" /> <!-- 开启延迟加载 -->
5 </settings>
```

原理

1. 使用 **CGLIB** 创建目标对象的代理对象
2. 当调用目标方法 `user.getOrderList()` 时，进入拦截器 `invoke` 方法，发现 `user.getOrderList()` 是 `null` 值，执行SQL查询order列表
3. 把order查询上来，然后调用 `user.setOrderList(List<Order> orderList)`，接着完成 `user.getOrderList()` 方法的调用



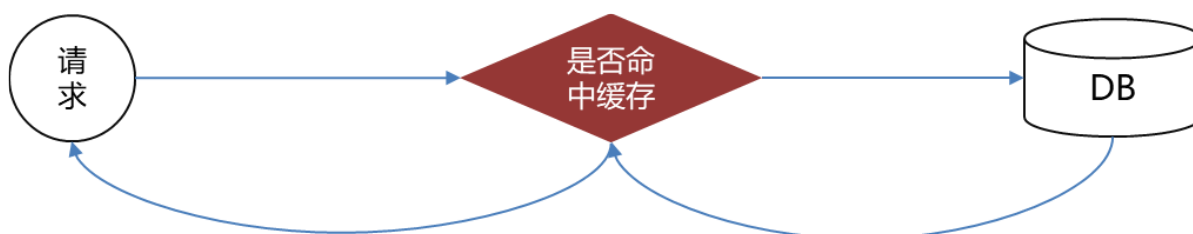
面试官：MyBatis是否支持延迟加载？

- 延迟加载的意思是：就是在需要用到数据时才进行加载，不需要用到数据时就不加载数据
- MyBatis支持一对一关联对象和一对多关联集合对象的延迟加载
- 在MyBatis配置文件中，可以配置是否启用延迟加载 `lazyLoadingEnabled=true|false`，默认是关闭的

面试官：延迟加载的底层原理知道吗？

1. 使用 **CGLIB** 创建目标对象的代理对象
2. 当调用目标方法时，进入拦截器 `invoke` 方法，发现目标方法是 `null` 值，执行SQL查询
3. 获取数据以后，调用 `set` 方法设置属性值，再继续查询目标方法，就有值了

一级、二级缓存



- 本地缓存，基于 **PerpetualCache**，本质是一个 **HashMap**
- 一级缓存：作用域是 **session** 级别
- 二级缓存：作用域是 **namespace** 和 **mapper** 的作用域，不依赖于 **session**

一级缓存

一级缓存：基于 **PerpetualCache** 的 **HashMap** 本地缓存，其存储作用域为 **Session**，当 **Session** 进行 **flush** 或 **close** 之后，该 **Session** 中的所有 **Cache** 就将清空，默认打开一级缓存

```
//2. 获取SqlSession对象，用它来执行sql
SqlSession sqlSession = sqlSessionFactory.openSession();
//3. 执行sql
//3.1 获取UserMapper接口的代理对象
UserMapper userMapper1 = sqlSession.getMapper(UserMapper.class);
UserMapper userMapper2 = sqlSession.getMapper(UserMapper.class);

User user = userMapper1.selectById(6);
System.out.println(user);

System.out.println("-----");
User user1 = userMapper2.selectById(6);
System.out.println(user1);
```

只会执行一次sql查询

二级缓存

二级缓存时基于 namespace 和 mapper 的作用域起作用的，不是依赖于 sqlSession，默认也是采用 PerpetualCache，HashMap 存储

```
//2. 获取SqlSession对象，用它来执行sql
SqlSession sqlSession1 = sqlSessionFactory.openSession();

//3. 执行sql
//3.1 获取UserMapper接口的代理对象
UserMapper userMapper1 = sqlSession1.getMapper(UserMapper.class);

User user1 = userMapper1.selectById(6);
System.out.println(user1);
sqlSession1.close();

SqlSession sqlSession2 = sqlSessionFactory.openSession();

System.out.println("-----");
UserMapper userMapper2 = sqlSession2.getMapper(UserMapper.class);
User user2 = userMapper2.selectById(6);
System.out.println(user2);

//4. 关闭资源
sqlSession2.close();
```

查询两次SQL

二级缓存默认是关闭的

开启方式，两步：

1. 全局配置文件

```
<settings>
  <setting name="cacheEnabled" value="true" />
</settings>
```


2. 映射文件

使用 `<cache/>` 标签让当前 `mapper` 生效二级缓存

注意事项：

1. 对于缓存数据更新机制，当某一个作用域（一级缓存 `Session` / 二级缓存 `namespace`）进行了新增、修改、删除操作后，默认该作用域下所有 `select` 中的缓存将被 `clear`
2. 二级缓存需要缓存的数据实现 `Serializable` 接口
3. 只有会话提交或者关闭以后，一级缓存中的数据才会转移到二级缓存中

面试官：MyBatis的一级、二级缓存用过吗？

- 一级缓存：基于 `PerpetualCache` 的 `HashMap` 本地缓存，其存储作用域为 `Session`，当 `Session` 进行 `flush` 或者 `close` 之后，该 `Session` 中的所有 `Cache` 就将清空，默认打开一级缓存
- 二级缓存是几乎 `namespace` 和 `mapper` 的作用域起作用的，不是依赖于 `SQL Session`，默认也是采用 `PerpetualCache`，`HashMap` 存储。需要单独开启，一个是核心配置 `[^`<setting name="cacheEnabled" value="true">`]`，一个是 `mapper` 映射文件 `[^`<cache/>`]`

面试官：MyBatis的二级缓存什么时候会清理缓存中的数据

当某一个作用域（一级缓存 `Session` / 二级缓存 `namespace`）进行了新增、修改、删除操作后，默认该作用域下所有的 `select` 中的缓存将被 `clear`。

框架篇面试题-参考回答

面试官：Spring框架中的单例bean是线程安全的吗？

候选人：

嗯！

不是线程安全的，是这样的

当多用户同时请求一个服务时，容器会给每一个请求分配一个线程，这是多个线程会并发执行该请求对应的业务逻辑（成员方法），如果该处理逻辑中有对该单例状态的修改（体现为该单例的成员属性），则必须考虑线程同步问题。

Spring框架并没有对单例bean进行任何多线程的封装处理。关于单例bean的线程安全和并发问题需要开发者自行去搞定。

比如：我们通常在项目中使用的Spring bean都是不可变的状态（比如Service类和DAO类），所以在某种程度上说Spring的单例bean是线程安全的。

如果你的bean有多种状态的话（比如 View Model对象），就需要自行保证线程安全。最浅显的解决办法就是将多态bean的作用由“`singleton`”变更为“`prototype`”。

面试官：什么是AOP

候选人：

aop是面向切面编程，在spring中用于将那些与业务无关，但却对多个对象产生影响的公共行为和逻辑，抽取公共模块复用，降低耦合，一般比如可以做为公共日志保存，事务处理等

面试官：你们项目中有没有使用到AOP

候选人：

我们当时在后台管理系统中，就是使用aop来记录了系统的操作日志

主要思路是这样的，使用aop中的环绕通知+切点表达式，这个表达式就是要找到要记录日志的方法，然后通过环绕通知的参数获取请求方法的参数，比如类信息、方法信息、注解、请求方式等，获取到这些参数以后，保存到数据库

面试官：Spring中的事务是如何实现的

候选人：

spring实现的事务本质就是aop完成，对方法前后进行拦截，在执行方法之前开启事务，在执行完目标方法之后根据执行情况提交或者回滚事务。

面试官：Spring中事务失效的场景有哪些

候选人：

嗯！这个在项目中之前遇到过，我想想啊

第一个，如果方法上异常捕获处理，自己处理了异常，没有抛出，就会导致事务失效，所以一般处理了异常以后，别忘了跑出去就行了

第二个，如果方法抛出检查异常，如果报错也会导致事务失效，最后在spring事务的注解上，就是@Transactional上配置rollbackFor属性为Exception，这样别管是什么异常，都会回滚事务

第三，我之前还遇到过一个，如果方法上不是public修饰的，也会导致事务失效

嗯，就能想起来那么多

面试官：Spring的bean的生命周期

候选人：

嗯！，这个步骤还是挺多的，我之前看过一些源码，它大概流程是这样的

首先会通过一个非常重要的类，叫做BeanDefinition获取bean的定义信息，这里面就封装了bean的所有信息，比如，类的全路径，是否是延迟加载，是否是单例等等这些信息

在创建bean的时候，第一步是调用构造函数实例化bean

第二步是bean的依赖注入，比如一些set方法注入，像平时开发用的@Autowired都是这一步完成

第三步是处理Aware接口，如果某一个bean实现了Aware接口就会重写方法执行

第四步是bean的后置处理器BeanPostProcessor，这个是前置处理器

第五步是初始化方法，比如实现了接口InitializingBean或者自定义了方法init-method标签或@PostConstruct

第六步是执行了bean的后置处理器BeanPostProcessor，主要是对bean进行增强，有可能在这里产生代理对象

最后一步是销毁bean

面试官：Spring中的循环引用

候选人：

嗯，好的，我来解释一下

循环依赖：循环依赖其实就是循环引用，也就是两个或两个以上的bean互相持有对方，最终形成闭环。比如A依赖于B，B依赖于A

循环依赖在spring中是允许存在，spring框架依据三级缓存已经解决了大部分的循环依赖

①一级缓存：单例池，缓存已经经历了完整的生命周期，已经初始化完成的bean对象

②二级缓存：缓存早期的bean对象（生命周期还没走完）

③三级缓存：缓存的是ObjectFactory，表示对象工厂，用来创建某个对象的

面试官：那具体解决流程清楚吗？

候选人：

第一，先实例A对象，同时会创建ObjectFactory对象存入三级缓存singletonFactories

第二，A在初始化的时候需要B对象，这个走B的创建的逻辑

第三，B实例化完成，也会创建ObjectFactory对象存入三级缓存singletonFactories

第四，B需要注入A，通过三级缓存中获取ObjectFactory来生成一个A的对象同时存入二级缓存，这个是有两种情况，一个是可能是A的普通对象，另外一个A的代理对象，都可以让ObjectFactory来生产对应的对象，这也是三级缓存的关键

第五，B通过从通过二级缓存earlySingletonObjects 获得到A的对象后可以正常注入，B创建成功，存入一级缓存singletonObjects

第六，回到A对象初始化，因为B对象已经创建完成，则可以直接注入B，A创建成功存入一级缓存singletonObjects

第七，二级缓存中的临时对象A清除

面试官：构造方法出现了循环依赖怎么解决？

候选人：

由于bean的生命周期中构造函数是第一个执行的，spring框架并不能解决构造函数的依赖注入，可以使用@Lazy懒加载，什么时候需要对象再进行bean对象的创建

面试官：SpringMVC的执行流程知道嘛

候选人：

嗯，这个知道的，它分了好多步骤

- 1、用户发送出请求到前端控制器DispatcherServlet，这是一个调度中心
- 2、DispatcherServlet收到请求调用HandlerMapping（处理器映射器）。
- 3、HandlerMapping找到具体的处理器（可查找xml配置或注解配置），生成处理器对象及处理器拦截器（如果有），再一起返回给DispatcherServlet。
- 4、DispatcherServlet调用HandlerAdapter（处理器适配器）。
- 5、HandlerAdapter经过适配调用具体的处理器（Handler/Controller）。
- 6、Controller执行完成返回ModelAndView对象。
- 7、HandlerAdapter将Controller执行结果ModelAndView返回给DispatcherServlet。
- 8、DispatcherServlet将ModelAndView传给ViewResolver（视图解析器）。
- 9、ViewResolver解析后返回具体View（视图）。
- 10、DispatcherServlet根据View进行渲染视图（即将模型数据填充至视图中）。
- 11、DispatcherServlet响应用户。

当然现在的开发，基本都是前后端分离的开发的，并没有视图这些，一般都是handler中使用Response直接结果返回

面试官：Springboot自动配置原理

候选人：

嗯，好的，它是这样的。

在Spring Boot项目中的引导类上有一个注解@SpringBootApplication，这个注解是对三个注解进行了封装，分别是：

- @SpringBootConfiguration
- @EnableAutoConfiguration
- @ComponentScan

其中@EnableAutoConfiguration是实现自动化配置的核心注解。

该注解通过@Import注解导入对应的配置选择器。关键的是内部就是读取了该项目和该项目引用的Jar包的的classpath路径下META-INF/spring.factories文件中的所配置的类的全类名。

在这些配置类中所定义的Bean会根据条件注解所指定的条件来决定是否需要将其导入到Spring容器中。

一般条件判断会有像@ConditionalOnClass这样的注解，判断是否有对应的class文件，如果有则加载该类，把这个配置类的所有的Bean放入spring容器中使用。

面试官：Spring 的常见注解有哪些？

候选人：

嗯，这个就很多了

第一类是：声明bean，有@Component、@Service、@Repository、@Controller

第二类是：依赖注入相关的，有@Autowired、@Qualifier、@Resource

第三类是：设置作用域 @Scope

第四类是：spring配置相关的，比如@Configuration，@ComponentScan 和 @Bean

第五类是：跟aop相关做增强的注解 @Aspect, @Before, @After, @Around, @Pointcut

面试官：SpringMVC常见的注解有哪些？

候选人：

嗯，这个也很多的

有@RequestMapping：用于映射请求路径；

@RequestBody：注解实现接收http请求的json数据，将json转换为java对象；

@RequestParam：指定请求参数的名称；

@PathVariable：从请求路径下中获取请求参数(/user/{id})，传递给方法的形式参数；

@ResponseBody：注解实现将controller方法返回对象转化为json对象响应给客户端。

@RequestHeader：获取指定的请求头数据，还有像@PostMapping、@GetMapping这些。

面试官：Springboot常见注解有哪些？

候选人：

嗯~~

Spring Boot的核心注解是@SpringBootApplication，他由几个注解组成：

- @SpringBootConfiguration：组合了- @Configuration注解，实现配置文件的功能；
- @EnableAutoConfiguration：打开自动配置的功能，也可以关闭某个自动配置的选项
- @ComponentScan：Spring组件扫描

面试官：MyBatis执行流程

候选人：

好，这个知道的，不过步骤也很多

①读取MyBatis配置文件：mybatis-config.xml加载运行环境和映射文件

②构造会话工厂SqlSessionFactory，一个项目只需要一个，单例的，一般由spring进行管理

③会话工厂创建SqlSession对象，这里面就含了执行SQL语句的所有方法

④操作数据库的接口，Executor执行器，同时负责查询缓存的维护

⑤Executor接口的执行方法中有一个MappedStatement类型的参数，封装了映射信息

⑥输入参数映射

⑦输出结果映射

面试官：Mybatis是否支持延迟加载？

候选人：

是支持的~

延迟加载的意思是：就是在需要用到数据时才进行加载，不需要用到数据时就不加载数据。

Mybatis支持一对一关联对象和一对多关联集合对象的延迟加载

在Mybatis配置文件中，可以配置是否启用延迟加载lazyLoadingEnabled=true|false，默认是关闭的

面试官：延迟加载的底层原理知道吗？

候选人：

嗯，我想想啊

延迟加载在底层主要使用的CGLIB动态代理完成的

第一是，使用CGLIB创建目标对象的代理对象，这里的目标对象就是开启了延迟加载的mapper

第二个是当调用目标方法时，进入拦截器invoke方法，发现目标方法是null值，再执行sql查询

第三个是获取数据以后，调用set方法设置属性值，再继续查询目标方法，就有值了

面试官：Mybatis的一级、二级缓存用过吗？

候选人：

嗯~，用过的

mybatis的一级缓存：基于 PerpetualCache 的 HashMap 本地缓存，其存储作用域为 Session，当Session进行flush或close之后，该Session中的所有Cache就将清空，默认打开一级缓存

关于二级缓存需要单独开启

二级缓存是基于namespace和mapper的作用域起作用的，不是依赖于SQL session，默认也是采用 PerpetualCache，HashMap 存储。

如果想要开启二级缓存需要在全局配置文件和映射文件中开启配置才行。

面试官：Mybatis的二级缓存什么时候会清理缓存中的数据

候选人：

嗯！！

当某一个作用域(一级缓存 Session/二级缓存Namespaces)的进行了新增、修改、删除操作后，默认该作用域下所有 select 中的缓存将被 clear。